SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal	
Instructor(s)Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpulasumalatha Mr. Bingi Raju	

Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	Week10 - Tuesday	**Time(s)**	
Duration	2 Hours	**Applicable to Batches**	23CSBTB01 To 23CSBTB52

AssignmentNumber:20.2(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	**Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code** **Lab Objectives:** • Understand how to test AI-generated code for common security vulnerabilities. • Learn to apply secure coding principles while analyzing AI outputs. • Practice detecting risks such as **SQL injection, XSS, hardcoded credentials, and weak encryption**. • Enhance code reliability and safety by using AI for secure refactoring.	Week10 - Tuesday
	Task 1 – Password Security Enhancement **Task:**	

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
- Refractor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Name: Shailaja Kanukuntla
Ht.no:2303A52155
Batch:34

Task 1: Password Security Enhancement

Prompt: Generate a Python user registration and login system that initially stores passwords in plain text and then refactor it to use secure password hashing. Ensure the updated program securely stores passwords and validates user login credentials.

Code:

Insecure Code (Plain Text – For Analysis)

```python
# Insecure version (plain text password storage)

users = {}

def register():
    username = input("Enter username: ")
    password = input("Enter password: ")
    users[username] = password
    print("User registered successfully!")

def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if users.get(username) == password:
        print("Login successful!")
    else:
        print("Invalid credentials!")
```

Secure Code (Using Hashing)

```python
import hashlib

users = {}

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def register():
    username = input("Enter username: ")
    password = input("Enter password: ")

    hashed_password = hash_password(password)
    users[username] = hashed_password

    print("User registered securely!")

def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    hashed_password = hash_password(password)

    if users.get(username) == hashed_password:
        print("Login successful!")
    else:
        print("Invalid credentials!")

register()
login()
```

Sample Input/Output:

Insecure Code Output (Plain Text – For Analysis)

```
Enter username: 2164
Enter password: 12345
User registered successfully!
Enter username: 2164
Enter password: 12345
Login successful!
```

Secure Code Output(Using Hashing)

```
Enter username: 2164
Enter password: 1234
User registered securely!
Enter username: 2164
Enter password: 1234
Login successful!
```

Code Explanation:

1. This task focuses on improving password security in an AI-generated program.
2. The original code stores passwords in plain text, which is highly insecure.
3. Plain-text passwords can be easily exposed if the system is compromised.
4. The improved version uses SHA-256 hashing from the hashlib library.
5. Passwords are converted into hashed values before storage.
6. During login, the entered password is hashed and compared securely.
7. This ensures that actual passwords are never stored or exposed.
8. The updated program enhances security and follows best practices.

Task 2: Secure API Key Management

Prompt:

Generate a Python program that initially uses a hardcoded API key and then refactor it to securely manage API keys using environment variables. Ensure the updated version prevents API key exposure and follows secure coding practices.

Code:

Insecure Code (Working Version)

```python
import requests

# Hardcoded API key (INSECURE)
API_KEY = "demo123"

try:
    # Using real public API (JSONPlaceholder)
    url = f"https://jsonplaceholder.typicode.com/posts/1?apikey={API_KEY}"

    response = requests.get(url, timeout=5)
    response.raise_for_status()

    print("Data fetched (Insecure):")
    print(response.json())

except requests.exceptions.RequestException as e:
    print("❌ API Error:", e)
```

Secure Code (Working Version)

```python
import requests
import os

# Set environment variable (for Colab testing)
os.environ["API_KEY"] = "demo123"

# Get API key securely
API_KEY = os.getenv("API_KEY")

if not API_KEY:
    print("❌ API key missing")
else:
    try:
        url = f"https://jsonplaceholder.typicode.com/posts/1?apikey={API_KEY}"

        response = requests.get(url, timeout=5)
        response.raise_for_status()

        print("Data fetched (Secure):")
        print(response.json())

    except requests.exceptions.RequestException as e:
        print("❌ API Error:", e)
```

Sample Input/Output:

Insecure Code Output(Working Version)

```
Data fetched (Insecure):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas to
```

Secure Code Output(Working Version)

```
Data fetched (Secure):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas to
```

Code Explanation:

1. This task focuses on securing API keys in application development.
2. The insecure version exposes the API key directly in the source code.
3. This makes the system vulnerable to misuse and unauthorized access.
4. The secure version uses environment variables to store sensitive data.
5. The API key is retrieved dynamically using the os module.
6. This prevents exposure of credentials in the codebase.
7. Error handling ensures the program works safely if the key is missing.
8. This approach follows industry best practices for secure API usage.

Task 3: Weak Encryption Detection

Prompt:

Generate a Python program that performs encryption and decryption using a basic method and then improve it using a stronger encryption technique. Ensure the improved version securely protects data and prevents easy decryption without the correct key.

Code:

Insecure Code (Weak Encryption – Base64)

```python
import base64

# Original message
message = "Hello123"

# Weak encryption (encoding)
encoded = base64.b64encode(message.encode()).decode()

print("Encrypted (Weak):", encoded)

# Decryption (very easy)
decoded = base64.b64decode(encoded.encode()).decode()

print("Decrypted:", decoded)
```

Secure Code (Strong Encryption – Fernet)

```python
from cryptography.fernet import Fernet

# Generate secure key
key = Fernet.generate_key()
cipher = Fernet(key)

# Original message
message = "Hello123"

# Encrypt
encrypted = cipher.encrypt(message.encode())
print("Encrypted (Secure):", encrypted)

# Decrypt (only with correct key)
decrypted = cipher.decrypt(encrypted).decode()
print("Decrypted:", decrypted)
```

Sample Input/Output:

Insecure Code Output(Weak Encryption – Base64)

```
Encrypted (Weak): SGVsbG8xMjM=
Decrypted: Hello123
```

Secure Code Output(Strong Encryption – Fernet)

```
Encrypted (Secure): b'gAAAAABp1SesT9-XXnH1HbIGu6iKn7uCRx-jGeX1E5MUcdQJzosmJCSOsK4BJW0F7p4rH-F8x5cH31zpE11SIfUMja0xJNh3aQ=='
Decrypted: Hello123
```

Code Explanation:

1. This task focuses on identifying weak encryption techniques.
2. The initial program uses Base64 encoding, which is not secure encryption.
3. Base64 can be easily reversed without any key, making it insecure.
4. This exposes sensitive data to unauthorized access.
5. The improved version uses the Fernet module from the cryptography library.
6. Fernet provides strong symmetric encryption with a secret key.
7. Data can only be decrypted using the same key, ensuring security.
8. This enhances confidentiality and follows modern cryptographic practices.

Task 4: Real-Time Application: Security Review of AI-Generated Web Service

Prompt:

Generate a simple Flask-based web service for user login and then perform a security review to identify vulnerabilities. Refactor the code using secure coding practices and implement fixes to improve security.

Code:

Insecure Code (AI-Generated Web Service)

```python
from flask import Flask, request

app = Flask(__name__)

users = {"admin": "12345"}  # Plain text password

@app.route('/login', methods=['POST'])
def login():
    username = request.json.get("username")
    password = request.json.get("password")

    if users.get(username) == password:
        return {"message": "Login successful"}
    else:
        return {"message": "Invalid credentials"}

app.run()
```

Secure Code (Improved Version)

```python
from flask import Flask, request, jsonify
import hashlib

app = Flask(__name__)

# Secure password storage (hashed)
users = {
    "admin": hashlib.sha256("12345".encode()).hexdigest()
}

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    # Input validation
    if not data or "username" not in data or "password" not in data:
        return jsonify({"error": "Invalid input"}), 400

    username = data["username"]
    password = hash_password(data["password"])

    if users.get(username) == password:
        return jsonify({"message": "Login successful"})
    else:
        return jsonify({"error": "Invalid credentials"}), 401

app.run()
```

Sample Input/Output:

Insecure Code Output (AI-Generated Web Service)

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

Secure Code Output (Improved Version)

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

Code Explanation:

1. This task involves reviewing an AI-generated web service for security flaws.
2. The original code stores passwords in plain text, making it insecure.
3. It lacks input validation, allowing potentially harmful inputs.
4. There is no encryption or hashing for password protection.
5. The improved version uses SHA-256 hashing for secure password storage.
6. Input validation ensures only proper data is processed.
7. Error handling improves reliability and prevents misuse.
8. These changes enhance overall application security and follow best practices.

Task 5: Exception Handling Improvement

Prompt:

Generate a Python program that takes user input and performs arithmetic operations without handling errors, and then improve it using proper exception handling. Ensure the improved version prevents crashes and displays meaningful error messages for invalid inputs.

Code:

Insecure Code (No Exception Handling)

```python
# Simple calculator (INSECURE)

num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

result = num1 / num2

print("Result:", result)
```

Secure Code (With Exception Handling)

```python
# Improved calculator with exception handling

try:
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    result = num1 / num2

    print("✅ Result:", result)

except ValueError:
    print("❌ Error: Please enter valid numeric values")

except ZeroDivisionError:
    print("❌ Error: Division by zero is not allowed")

except Exception as e:
    print("❌ Unexpected Error:", e)
```

Sample Input/Output:

Insecure Code Output(No Exception Handling)

```
Enter first number: 1
Enter second number: 2
Result: 0.5
```

Secure Code Output(With Exception Handling)

```
Enter first number: 5
Enter second number: 7
✅ Result: 0.7142857142857143
```

Code Explanation:

1. This task focuses on improving program reliability using exception handling.
2. The original program does not handle runtime errors, causing crashes.
3. Invalid inputs such as text instead of numbers lead to ValueError.
4. Division by zero results in ZeroDivisionError.
5. The improved version uses try-except blocks to handle these errors.
6. Specific exceptions provide clear and user-friendly messages.
7. A general exception handler ensures unexpected errors are also managed.
8. This enhances program stability and improves user experience.